

Neural Network Pipelines for Pulse Pile-Up Classification and Recovery in Organic Scintillators

Grace Feng¹, Mehak Chohan²

¹Computer Science, University of Michigan, Ann Arbor, U.S.A

²Computer Science, University of Michigan, Ann Arbor, U.S.A

Abstract

Current analytical methods for the identification of neutron and gamma pulses from organic scintillators are not able to reliably classify piled-up pulses, resulting in these pulses being discarded completely, decreasing measurement efficiency in high-rate environments. In this work, we explore the implementation of neural networks as an alternate approach for pileup classification and recovery. After translating a Low Compute Pulse Pileup Recovery network in MatLab to a Python-based TensorFlow equivalent, a complexity analysis of two networks - High Compute and Low Compute -revealed an advantage of the Low Compute model in producing compatible accuracy for real-time classification at a fraction of the computational cost. While the Low Compute network produced good accuracy, this was out-performed by the High Compute network for pileups with close separation (< 4 ns) between waveforms. Preliminary efforts to test the models aligned with previous analytical results, showing promise for improved particle discrimination for close-pileups on more structured raw inputs.

1. INTRODUCTION

Current pulse shape discrimination (PSD) analysis via the charge integration method use a fixed particle-discrimination line to distinguish between neutron and gamma pulses. These conventional methods either discard pile-up events, throwing away real particle signals, or misclassify piled-up photon events as neutrons, overestimating the apparent neutron rate by as much as 27% [2]. This results in the loss of valuable information, yielding unreliable results. Our project goal is to recover information from piled-up waveforms and correctly recover particle classification from the constituent waves. Through this work, insight into exclusive advantages of previous neural network based pipelines was gained. The neural network approach shows promise for classifying on real piled-up detector data.

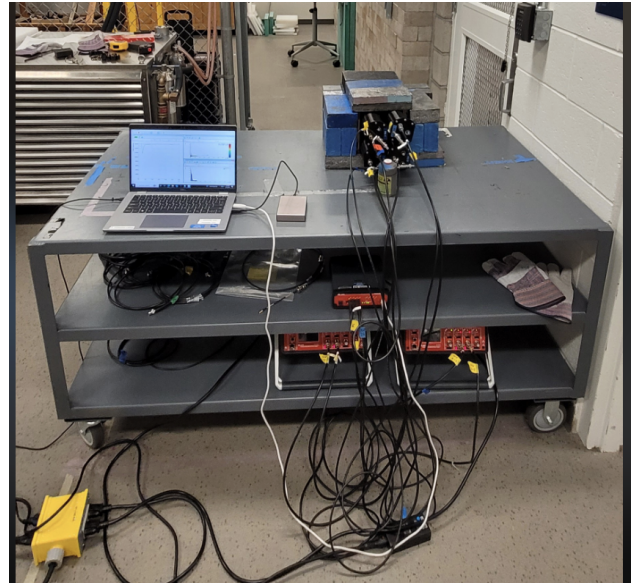


Figure 1. Shield Cave used to obtain raw waveform data

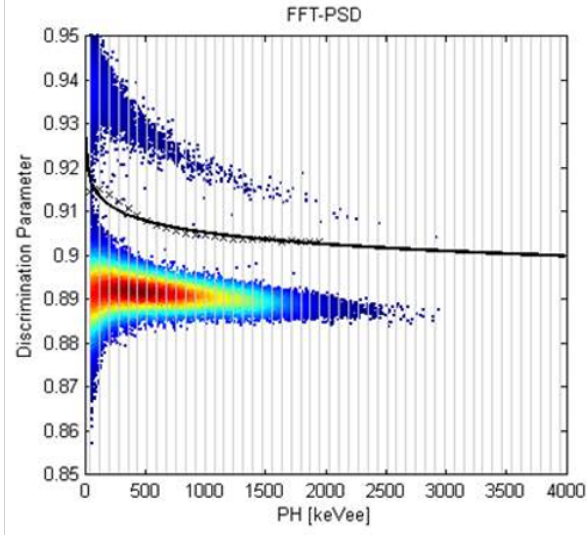


Figure 2. Analytical pulse shape discrimination line used to differentiate neutrons from photons

2. METHODOLOGY AND DATA

We adapted the pipeline described by Jinia et al. in Ref.[1] from MATLAB into Python to have an accurate comparison with the pipeline by McHugh and Beyer. An analysis of the two original pipelines is described below.

2.1. Overview of the Original Architecture

The Low Compute pipeline consists of six stacked feedforward neural networks operating in sequence. Derived from the architecture described by Jinia et al. [1], pre-processed waveforms first pass through *Classify Top*, which sorts each waveform into one of two categories: single pulse or piled-up (double) pulse. Since our aim was to conduct a direct comparison with the high compute pipeline, we removed the classification bucket for triple/quadruple events to match the structure of the other network. Single-pulse events pass through *Cleanser I*, an autoencoder-based check that independently verifies the single-pulse classification and reclassifies any pulse it flags as noisy back into the pile-up branch; surviving single pulses are then passed to *Classify Singles*, which assigns a final neutron/gamma label. Piled-up events pass through *Cleanser II*, which separates poor-signal-to-noise pile-up events from “true” pile-ups; true pile-ups are passed to *Classify Pile-Up*, which routes each event to one of three sub-classifiers – *Classify*

Close, *Classify Split*, or *Classify Cut* – based on the time separation between the two constituent pulses. Each sub-classifier then determines the neutron/gamma composition of the pair.

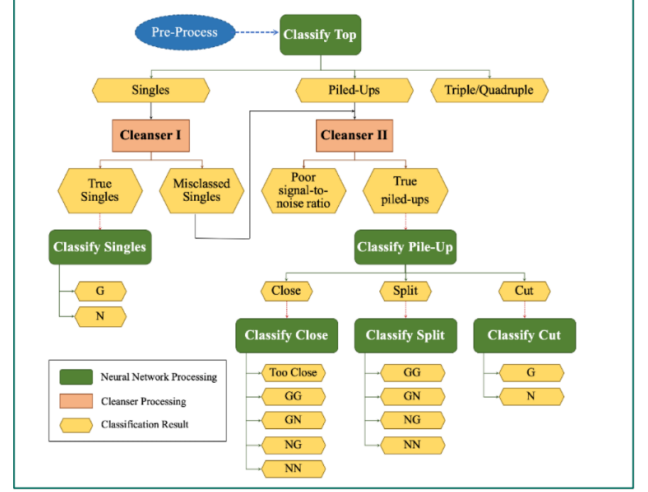


Figure 3. Six Neural Network pipeline from Jinia et al. [1]

During translation from MATLAB to Python, two discrepancies in implementation were identified and corrected.

- **Double softmax activation.** The initial translation applied a softmax activation at the network’s output layer in addition to the softmax already applied internally by the cross-entropy loss function, distorting the gradient during training. The redundant activation was removed.
- **Missing sparsity regularization.** MATLAB’s `trainAutoencoder` function applies a Kullback–Leibler divergence–based sparsity penalty by default. This was absent from the initial Python translation, causing training loss to plateau; adding it brought training behavior back in line with the original MATLAB implementation.
- **Too Close Section Exclusion** The pipeline previously did not attempt to classify pulses deemed “Too Close”, corresponding to interpulse delays of 0-60 ns, as the accuracy was expected to plummet. However, because the other pipeline attempted to classify such pulses, the training code was instructed to treat “Too Close” pulses as “Close” pulses and attempt to classify the particles within them. A comparison was then made on

the accuracy of "Too Close" type pileup classification.

2.1.1 High Compute Pipeline

The High Compute pipeline begins with a single classifier, *C1*, that separates single pulses from pile-up events using a bidirectional long short-term memory (BiLSTM) network. A bidirectional architecture processes the waveform from both the leading and trailing ends simultaneously, allowing the network to learn relationships between a pulse's peak and its tail rather than only relying on features local to the rising edge. There are two multitask autoencoders, used for feature learning, trained on single or pile up pulses that then return the pulse classifications. Unlike the architecture described by Jinia et al. [2], this method does not differentiate between close, cut, and split type pulses. Instead all pileups are classified with one network.

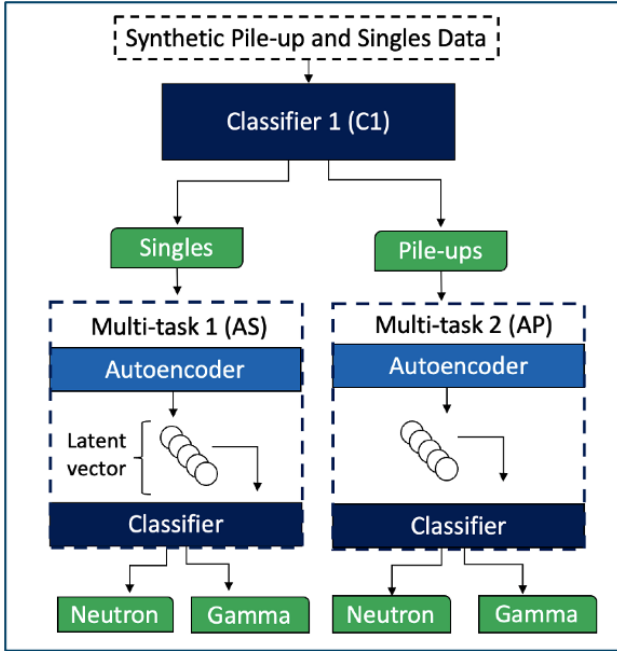


Figure 4. Diagram of BiLSTM autoencoder training pipeline proposed by McHugh and Beyer in Ref. [4]

Training data for the Low Compute pipeline was edge aligned, while High Compute pipeline was not. We opted to keep both training datasets not edge aligned, instead adopting inter-peak delays.

2.2. Synthetic Data Generation

Because no ground-truth-labeled dataset of real piled-up waveforms exists, both pipelines were trained and evaluated on a common synthetic pile-up dataset constructed from a shared pool of high-confidence single neutron and gamma pulses, obtained through time-of-flight measurements. The waveform generator was adapted from the high compute pipeline's previous generator. Standardizing the data generation was important in allowing the accuracy and complexity comparison to reflect architectural differences rather than differences in training data.

We set aside 50% of the raw waveforms to be true singles. Baseline subtraction and euclidean normalization were also performed, as done in the pipeline described by McHugh and Beyer. Synthetic pile-up events were generated by summing randomly paired single pulses at a randomly sampled delay, drawn uniformly from 0–200 ns. Two representative examples illustrate our approach: at a 6 ns delay, the primary and secondary peaks, occurring at samples 32 and 38, are barely resolved and overlap substantially in the waveform's rising edge; at a 50 ns delay, the two peaks are cleanly separated and the pair more closely resembles two independent single pulses. Our implementation refined the delay as the peak-to-peak distance between the two constituent pulses, rather than the distance from the beginning of the sample, since the peaks for the first delay were not edge aligned.

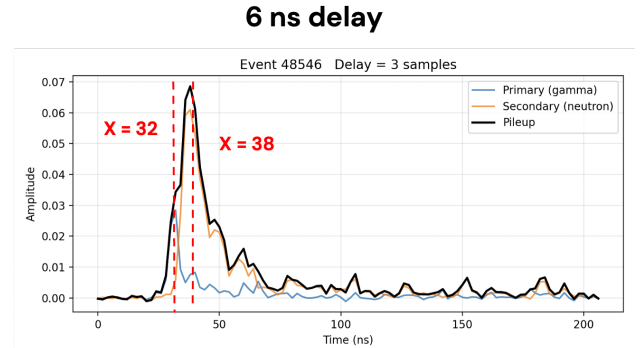


Figure 5. Example of a synthetically generated pulse with a 6 ns inter-peak delay

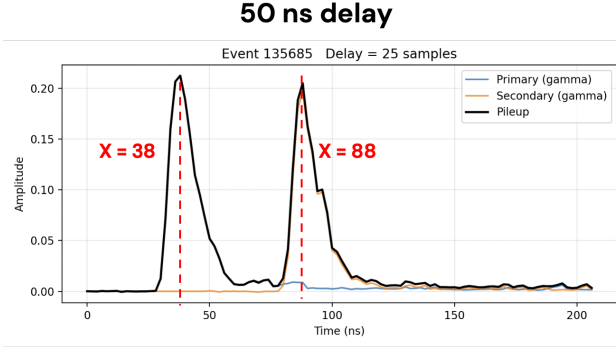


Figure 6. Example of a synthetically generated pulse with a 50 ns inter-pulse delay

Then we trained the two pipelines on the same synthetic dataset, with a 60/20/20 split between training, validation, and testing.

2.3. Complexity Analysis

To assess feasibility for real-time, on board deployment, we quantified computational complexity as the number of multiply-accumulate (MAC) operations required per pulse at inference time. The low compute pipeline consists of 6 fully connected neural networks, so we were able to sum up the number of multiplications that occur at inference by looking at the size of each layer. The high compute pipeline used more complicated architectures; by referencing documentation about the input parameters, we were able to determine the complexity of each of these elements and sum them for each stage in the pipeline.

The High Compute total is the exact sum of its three listed stages. The Low Compute total (9,199) exceeds the sum of its three listed stages ($3,390 + 3,360 + 339 = 7,089$) by 2,110 MACs/pulse; this remainder is attributable to the additional Cleanser I/II and Close/Split/Cut sub-classifier stages present in the six-network Low Compute pipeline. Based on total inference cost, the Low Compute pipeline requires approximately $66,445,792 / 9,199 \approx 7,225 \times$ fewer MAC operations per pulse than the High Compute pipeline, and completes evaluation roughly $2 \times$ faster (17 s vs. 36 s) on the same hardware. These timing figures were measured on the Great Lakes cluster, which has substantial compute and memory resources relative to an embedded target. In an FPGA

environment, where both compute throughput and on-chip memory are far more constrained, this gap would be expected to widen.

2.4. Real Data Testing

Both pipelines were developed and compared using synthetic data, but the ultimate goal of this work is classification on a real measurement data. We tested the trained High Compute pipeline against a real active-interrogation measurement in which a Linac-driven photon beam irradiated a 5.23 depleted uranium cube (see Fig. 1). An example of a raw waveform measured by this device is depicted by Fig. 7). These waveforms were preprocessed in the same way as the training dataset for consistency.

The existing analytical method for this measurement rejects pile-up using a fixed amplitude threshold ($0.1 \times$ the maximum pulse amplitude) applied after Savitzky-Golay smoothing: any waveform with more than one detected peak above threshold is discarded outright, with no attempt made to recover its neutron/gamma content. This threshold is sensitive to noise level. A single pulse with noise fluctuations crossing $0.1 \times$ can register spurious extra peaks and be rejected as pile-up, which is the practical motivation for testing an ML-based alternative on this same data.

Real waveforms were acquired at 152 samples per pulse. Since the trained pipelines expect 104-sample input, our options were 1) to retrain the pipeline on synthetic data acquired at 152-samples per pulse or 2) essentially chop the real measurements to 104 samples as per our pipelines expected input.

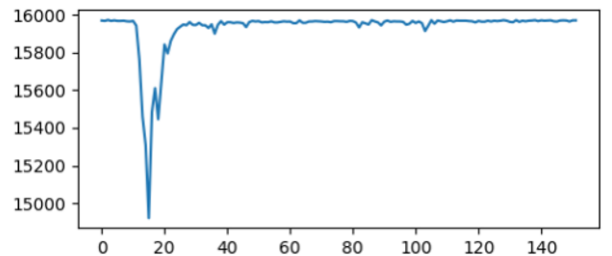


Figure 7. Raw waveform from Linac-driven photon beam

To decide, we ran the BinReader pile-up filter across the full real dataset. This flagged 25,655

Table 1. Complexity analysis (MACs per pulse) and evaluation time on cluster.

Pipeline	C1 / Classify Top	A1 / Classify singles	A2 / Classify pileup	Total	Compute time on cluster
High Compute	13,692,992 MACs/pulse	6,483,232 MACs/pulse	46,269,568 MACs/pulse	66,445,792 MACs/pulse	36 seconds
Low Compute	3,390 MACs/pulse	3,360 MACs/pulse	339 MACs/pulse	9,199 MACs/pulse	17 seconds

events as pile-up, a rate of 9.0%. Of those flagged events, 32.6% had their second pulse landing past sample 104, meaning chopping the data to 104 samples would lose real pile-up information from roughly 3% of all accepted events. Because of time constraints, we chose to chop the real waveforms to 104 samples rather than retrain on the full 152-sample window. However, we added an optional padding function to the current waveform preprocessing script, allowing for retraining of the networks on pulse of 104 samples or more.

2.4.1 Previous Analytical Method

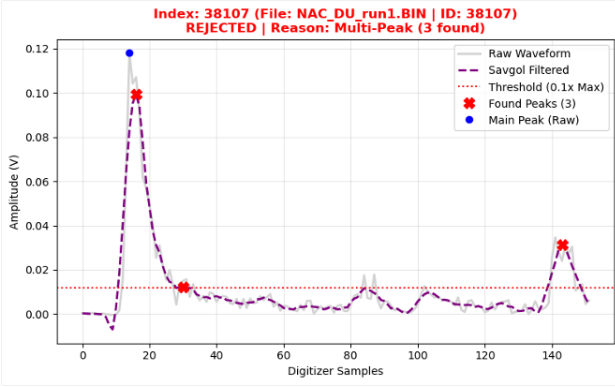


Figure 8. Example of a rejected pulse at a 0.01V threshold. Note the second marked X; this is most likely not another pulse, but noisiness being misconstrued as another peak [3].

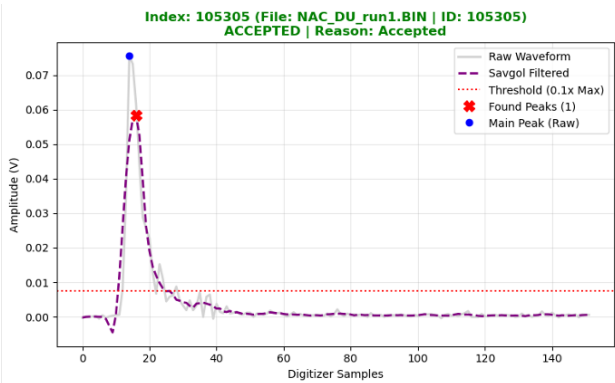


Figure 9. Example of a pulse that would be accepted with this method. Only one peak was located above the threshold [3].

2.4.2 Effect of Savgol Filter on Delays Distribution

After the first run of the neural network, we noticed that 80% of pulses were being classified as pileups. Upon further inspection, we realized these flagged pileups were actually singles with noisy tails. By applying a Savitzky-Golay filter (Savgol), we were able to reduce this effect, as evidenced in Figures 10 and 11.

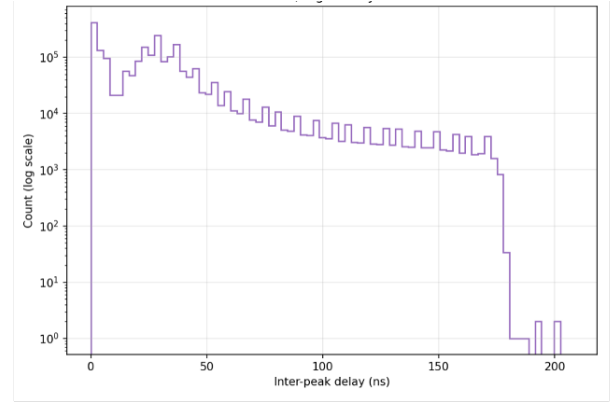


Figure 10. Counts at various inter-peak delays for classified raw data pileups without Savgol pre-processing.

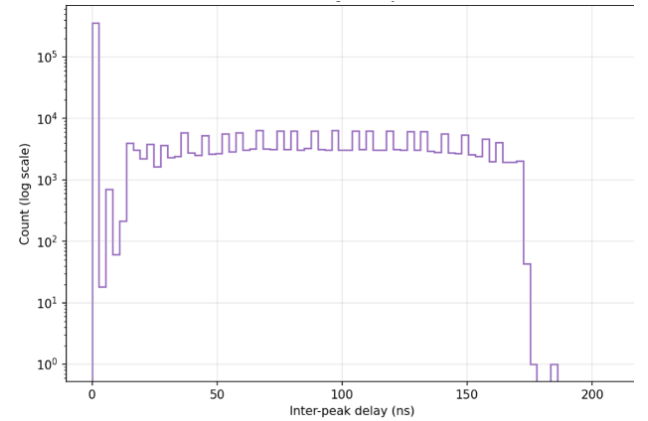


Figure 11. Counts at various inter-peak delays for classified raw data pileups with Savgol preprocessing. Note the more uniform distribution.

We expected to see a uniform distribution of delays between detected pileups, due to the independence of pulse occurrence. Due to observing a disproportionate number of close type pulses, we determined that the noisiness

of the pulse peaks was becoming misconstrued for a piled-up pulse. After applying the Savgol filter, the distribution of delays became much more uniform. This hypothesis was supported by the drastic decrease in detected pileups, from 80% to 18%, after the filter was applied. As evidenced in Figure 12, applying the filter kept the overall shape of the tail, while reducing the sharp noisy edges that were being misclassified as close pileups. Due to time constraints, we were not able to conduct a conclusive parameter search for optimal Savgol parameters, so we use the parameters that were being applied to the pulses in the analytical pipeline to facilitate a direct comparison. Conducting a thorough search for optimal parameters is recommended for future work, especially for noisy data.

Note the spike in pulses around the 0 ns delay bucket. A true pileup pulse with a 0 ns delay is difficult to distinguish from a true single. Because little was known about the expected pileup to single ratio within the raw data and due to a lack of ground truth labeling, it was difficult to make any assertions about the accuracy of the model. Two main explanations for this behavior arise: the model is correctly determining these pulses as true close pulses, uncovering particles otherwise rejected by analytical approaches or the model is confusing true singles for pileups. Running additional tests on more well defined waveforms produced by a high gamma source will guide the next steps of this research.

2.4.3 Examples of Denoised Pulses

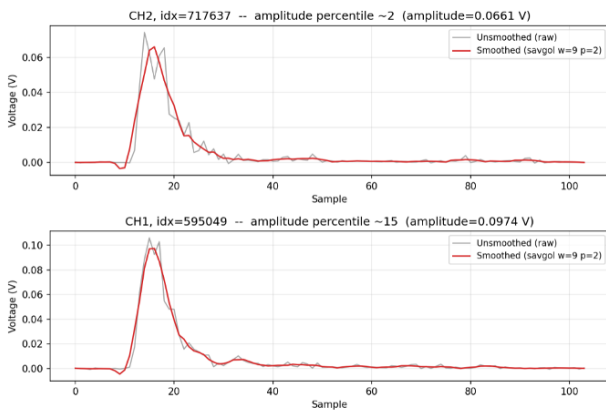


Figure 12. Examples of pulses with Savgol filter applied.

3. RESULTS AND DISCUSSION

3.1. Pipeline Comparison

Accuracy as a function of inter-peak delay was computed for both pipelines to characterize how performance degrades as pile-up events become more closely spaced. The low compute pipeline struggled more with classifying close-type pileups, but achieved a comparable accuracy to the high compute pipeline as delays increased. As observed in Figures 13 and 14, the accuracy of pulses with an inter-peak delay falling between 0-60 ns was on average 20% less accurate than further spaced delays. On the low compute pipeline, for pulses of 0-4 ns delays, this result degrades to 50% accuracy, essentially no better than a coin flip.

3.1.1 Low Compute Pipeline

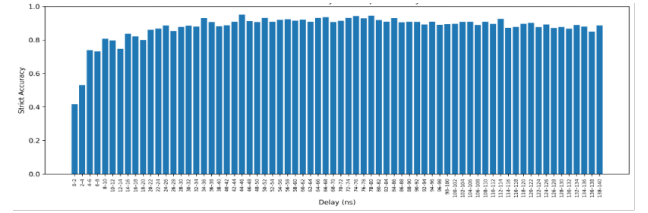


Figure 13. Accuracy in the low compute pile-up model for different inter-peak delays

3.1.2 High Compute Pipeline

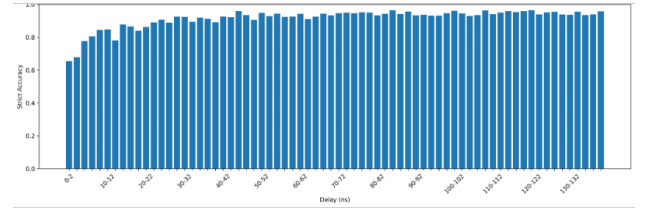


Figure 14. Accuracy in the high compute pile-up model for different inter-peak delays

Confusion matrices were generated for each classifier stage (C1, A1, and A2) in both pipelines, reporting the normalized fraction of true-label events assigned to each predicted class. C1 accuracy corresponds to the accuracy in classifying if the pulse is a single or a pileup. A1 accuracy refers to accuracy in classifying a routed single as neutron or gamma, and A2 accuracy refers to classifying both of the constituent pulses in a pileup accurately.

Both pipelines performed similarly for correctly routed singles and pileups (A1 and A2). The low compute pipeline had an A1 accuracy of 99.25% and the high compute had an accuracy of 99.45%. For A2 accuracy, the low and high compute pipelines had accuracies of 89.0% and 89.6% respectively. The most significant difference in accuracy was observed in C1 (singles vs. pileups). The low compute pipeline achieved an accuracy of 88.8%, a drop compared to the high compute's 97.55% accuracy. Specifically, true pile-ups are being misclassified as singles. This follows with the previous inter-peak delay analysis, showing that the low compute pipeline struggles to both differentiate singles and pileups as well as classify identified pileups when pulses are closer together. The increase in accuracy in the high compute is supported by increased complexity the BiLSTM and multi-task autoencoders that consist it. For applications that highly value information recovery and waveform reconstruction, the high compute pipeline emerged as the preference. However, because the low compute pipeline was able to achieve a comparable result at 7200x less computation per waveform, we were able to provide the models trained by this pipeline to FPGA student researchers to be run on lower-compute architectures.

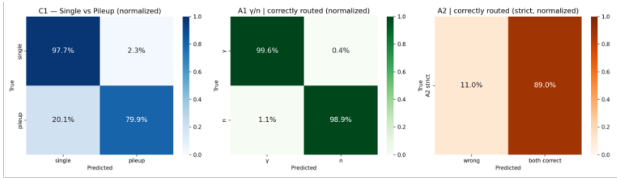


Figure 15. C1, A1, and A2 accuracy for the low compute pipeline

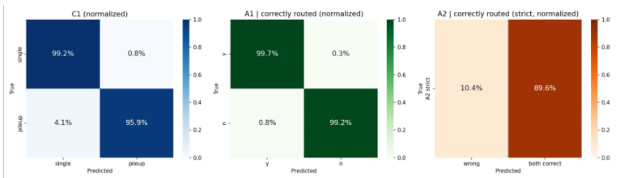


Figure 16. C1, A1, and A2 accuracy for the high compute pipeline

3.1.3 Real Data Inference

The pulse height distributions for the analytical method and neural network based method was determined for including pile-ups and then excluding pileups, as depicted in Figures 17 and

18. The current analytical method, which was based on an arbitrary voltage filter, estimated a count of 414,830 neutrons across the four channels and all waveforms. However, when pileups were discarded, this estimate fell to 93,666 neutrons. Two factors were proposed to explain this observation. The arbitrary threshold that determines whether or not a wave is a single or pile-up could be accidentally mistaking noisy singles for pileups. However, as seen in Figure 20, the single vs. pileup split did not change drastically between the two methods, indicating that the pulses were not in fact misclassified. Instead, significant portions of information were being lost through pileup discarding.

The result from running the trained networks revealed an increase in detected neutrons. However, this is likely an indication high overestimation rate of neutrons when including piled-up pulses. When considering all pulses, the neural network estimated that around 25% of the recovered pulses were from neutrons, a drastic increase from the analytical pipeline's 19%. However, even when only singles were classified, the neural network identified 11.9% of particles as neutrons as opposed to the 4.7% number provided by the analytical pipeline. When isolating the evaluation to just singles, the networks still disagreed on the classification of just over half of all pulses. Without access to ground truth labeling, it is difficult to determine whether the discrepancy falls in the analytical pipeline or the neural network.

Because the network was trained on different wave forms of differing lengths, it may have struggled to reach the same level of accuracy with this noisier data. Although the Savgol filter helped mitigate the noisiness, retraining the network on similar labeled data would help uncover this possible deficiency. To fully confirm our suspicions, additional tests will have to be run on well documented data and compared to the expected gamma/neutron split.

Another concern we had involved the photon PHD plot. Between including and discarding pile-up pulses, we would expect a similar increase in photons to emerge. The number of photons increases by 642718 photons detected, which is a greater increase than the 450289 increase

in neutrons, but because 85% of pulses were classified as photons, we would still expect to see a larger jump. As seen in Figure 11, there was a huge spike in detected piles with delays from 0-2 nanoseconds. These could be in reality true neutron singles, but because they are counted as pileups, they may be double counted. We suggest going forward with retraining the network to uncover the root cause of this discrepancy.

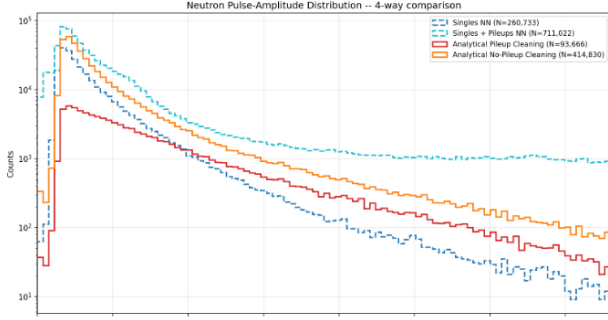


Figure 17. Pulse Height Distributions for Analytical and Neural Network Pipelines for Neutrons

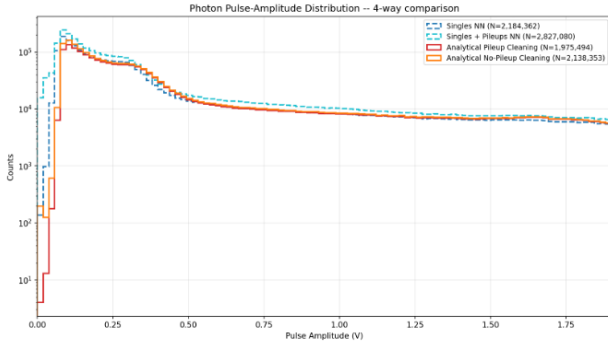


Figure 18. Pulse Height Distributions for Analytical and Neural Network Pipelines for Photons

4. CONCLUSIONS AND FUTURE WORK

Both ANN pipelines successfully classify piled-up neutron/photon events that traditional charge-integration PSD approaches would reject outright. The Low Compute pipeline achieves classification accuracy comparable to the High Compute pipeline at roughly $7,200\times$ lower inference cost, making it well suited to real-time, on-board FPGA deployment. The High Compute pipeline, while more computationally expensive, offers opportunities to uncover additional information beyond classification alone, most notably through waveform reconstruction of the individual constituent pulses in a pile-up event,

rather than only their neutron/gamma labels.

We applied the trained High Compute pipeline to a real active-interrogation measurement and found it produced results comparable to the current analytical approach, validating that the pipeline generalizes reasonably from synthetic training data to real detector waveforms. As part of this work, we also reorganized the source code for both the High Compute and Low Compute pipelines, consolidating them into a form intended to be usable and extensible for future work.

4.1. Limitations and Future Work

Triple pile-up events are most likely being folded into the general “pile-up” category by both pipelines, since neither currently has a dedicated triple/quadruple classification path; this likely skews the neutron/gamma composition results reported for pile-up events, since a genuine three-pulse event does not map cleanly onto a two-pulse classification scheme. Separating out and explicitly modeling triple pile-up events is a natural next step. Chopping real waveforms from 152 to 104 samples results in an estimated 3% loss of pile-up information. This is avoidable in principle by retraining on the full 152-sample window once a suitable labeled dataset exists. We were able to add padding capabilities to our synthetic waveform generation, thus producing synthetic labelled training data of 152 samples; however, due to time constraints, we were not able to retrain the networks and compare the results.

This work was unable to establish an independent ground truth for the real measurement, relying instead on comparing the ANN pipeline’s output against the existing analytical method rather than against a directly known neutron/gamma composition. For future work, we recommend measuring with a strong, well-characterized gamma source and using the known source composition as ground truth to directly measure each pipeline’s misclassification rate. This would prevent the need to estimate what the true neutron/gamma rates actually are, and would give a much stronger basis for evaluating real-data performance than the comparative approach used here.

5. DECLARATION

Generative AI Disclosure: We would like to acknowledge that generative AI (Claude, ChatGPT, UM-GPT) was used to assist in generating scripts, tests, and visually modifying graphics.

5.1. Acknowledgments

The authors would like to thank Dr. Abbas Jinia from the University of Michigan Detection for Nuclear Non-proliferation Group (DNNG) for supplying the time-of-flight waveform dataset used in this study for the generation of synthetic data. Additional thanks to Lorenzo Loi from the Politecnico di Milano for supplying us the measurement data for the active interrogation of depleted uranium, collected at the University of Michigan DNNG laboratory.

6. REFERENCES

- [1] Abbas J. Jinia et al. “An Artificial Neural Network System for Photon-Based Active Interrogation Applications”. In: *IEEE Access* 9 (2021), pp. 119871–119880. DOI: 10.1109/ACCESS.2021.3108406.
- [2] Abbas Johar Jinia. “Development of Neural-Network-Based Digital Pulse Processing for Photoneutron Detection”. PhD thesis. Ann Arbor, MI: University of Michigan, 2023.
- [3] Lorenzo Loi. *Data Processing Script for Depleted Uranium Measurement [Python Code]*.
- [4] Annamaria McHugh and Kurt Beyer. *Application of Machine Learning for the Recovery of Particle-type information from Pulse Pileup in Organic Scintillators*. Final Report for NERS-590-002. University of Michigan, 2026.